

AutoM2M: Checking the Composition of Automatically Assembled LLM Agent Teams

Abstract—[TODO: abstract]

Index Terms—LLM-based multi-agent systems, model transformations, architectural mismatch, failure attribution, code generation

I. Introduction

[TODO: intro]

II. Background

AutoM2M builds on AgentM2M [1], which treats every hand-off between two LLM agents as a model-to-model (M2M) transformation. We introduce its concepts on one running example and state them as definitions that Section III builds on.

A. Running example

DevTeam builds a small to-do service. The Analyst turns requests into user stories with acceptance criteria; the Architect designs one API operation per accepted story; the Tester writes executable tests; the Developer implements each operation. The requirements contain story S1 create a task (criterion S1.1: a title is required), S2 mark a task done (S2.1: completing a done task is rejected) and S3, a draft. In a team built by CaptainAgent or AutoAgents [2], [3], these four agents would be four role prompts and a plan in prose. In AgentM2M, each is a form, and each hand-off is a rule set.

B. Views, models and hybrid rules

Definition 1 (View metamodel, model). A view metamodel $MM = (C, F)$ is a set of classes C and typed features F : attributes with a primitive type, and references to classes (possibly of other views), each with a lower bound (mandatory if ≥ 1) and an upper bound. A model $M \in \mathcal{M}(MM)$ is a set of objects that conforms to MM .

Each agent owns one view, the form it fills in (Listing 1). The Architect’s Operation knows nothing about stories except through a reference, so each agent stays in its own vocabulary, as in view-based development [4], [5].

Listing 1. View metamodels of DevTeam (KM3-like).

```
package Req { class UserStory { attribute id, title,
    status : String;
    reference criteria[0-*] : Criterion; }
    class Criterion { attribute id, text :
    String;
    reference story[1] : UserStory; } }
package Arch { class Operation { attribute name, signature
    : String;
    reference story[1] : Req!UserStory; } }
package Test { class TestSuite { attribute name, code :
    String;
```

```
    reference story[1] : Req!UserStory; } }
package Code { class CodeEdit { attribute file, body :
    String;
    reference operation[1] : Arch!Operation;
    reference story[1] : Req!UserStory; } }
```

Definition 2 (Hybrid rule). A hybrid rule r has a source pattern (typed variables over source views), an optional guard, and a target pattern creating one object of class D . Its bindings are partitioned into structural bindings $f \leftarrow e$, where e is a navigation path or literal evaluated by a deterministic engine, and stochastic bindings $f \leftarrow @llm(p, fp)$ with a prompt p , a footprint fp (the set of navigation paths an LLM may read), and a validator chk that a sampled value must pass. A hand-off T is a set of rules with source views $src(T)$ and one target view $tgt(T)$.

Listing 2. Hand-off Arch2Code of DevTeam. @llm marks a value an LLM writes; @check its validator.

```
rule Op2Edit {
  from op : Arch!Operation, ts : Test!TestSuite (ts.story =
    op.story)
  to e : Code!CodeEdit (
    file <- op.name, operation <- op, story <- op.story,
    -- structural
    body <- @llm('Implement the operation',
      Sequence{op.signature, op.story.
        criteria}), -- footprint
    @check body.passes_tests(op.story, ts.code) ) }
    -- validator
```

The engine computes the structure of the hand-off: for every match of the source pattern it creates one target object, fills structural bindings and resolves references, as ATL’s matched rules do [6]. Only a value no formula can compute, here an implementation, is sampled from an LLM, which sees only the footprint. A sample is kept only if the validator accepts it; otherwise the engine re-samples, at most k times, and then escalates. Because Op2Edit matches every pair of an operation and its test suite, no operation can be forgotten by an LLM.

Definition 3 (Trace link, stamp, obligation). When rule r fires on match m and creates t , the engine records a trace link (m, t, r) [7], [8]. For every accepted stochastic value $t.f$ it records a stamp, the digest of the footprint’s value at acceptance time. A value whose stamp differs from the digest of its current footprint is an obligation.

Stamps make change propagation exact: when the Analyst edits S2.1, only the values whose footprints read S2.1 become obligations. For this paper, the important consequence is different: the complete input of every

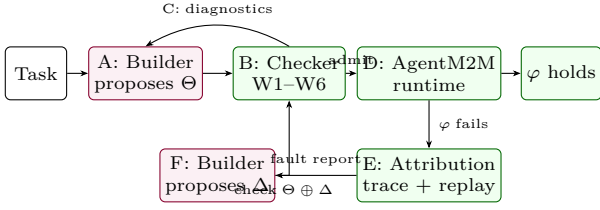


Fig. 1. AutoM2M. LLM steps (purple) propose; deterministic steps (green) decide. A team runs only after the checker admits it; every repair passes the same checker.

LLM-written value is recorded and small, which is the information that failure attribution needs [9].

Definition 4 (Team model, acceptance). A team model $\mathcal{T} = (A, V, \mathbf{T}, \omega, \varphi)$ has agents A , views V , hand-offs $\mathbf{T}$, write rights $\omega : A \rightarrow 2^V$, and an acceptance predicate φ evaluated by the engine: every match is covered by a trace link, no obligation or escalation is open, all stamps are fresh, and all validators pass.

The team is done when φ holds, not when an agent says so. Adding an agent at runtime is a higher-order transformation (HOT) [10] over $\mathcal{T}$, kept causally connected to the running team (models@run.time [11]).

C. What AgentM2M leaves open

AgentM2M’s team model, metamodels and rules are written by a person. Its evaluation on DevBench [12] named “learning the glue” as its first open problem. For an automatically assembled team, the team builder is the architect: it must write the forms, the rules and φ , and nothing guarantees that it writes good ones. The composition gap does not disappear; it moves from the messages to the team specification. The next section checks that specification.

III. Approach

AutoM2M lifts AgentM2M’s division of labour one level up. AgentM2M lets an LLM write values while a deterministic engine owns the structure of each hand-off. AutoM2M lets an LLM propose a team while a deterministic checker owns the decision to admit it (Figure 1). Algorithm 1 admits a team; Algorithm 2 runs it, attributes failures and repairs it.

A. Step A: typed teams

Definition 5 (Typed team). A typed team is $\Theta = (A, V, \mathbf{T}, \omega, \kappa, \tau, G, \varphi)$: a team model (Definition 4) whose views include a goal view MM_0 into which the task is lifted, extended with $\kappa : A \rightarrow 2^{Tools}$, the tools of each agent; τ , the tools each stochastic binding’s validator needs; and a set G of goal obligations (C, s, k) , a goal-view class C , a scope s and a kind $k \in \{checked, delivered\}$. Every validator comes from a library that declares its strength (form: parses, compiles; behaviour: executes the value against examples or tests) and its tool needs.

A builder fills a constrained JSON format, not free ATL: footprints are navigation paths and validators are library entries. Agents keep free-text role prompts; how an agent thinks remains prose, but what it receives and produces is typed. Listing 3 renders a first proposal for DevTeam that we wrote to contain one instance of each defect class.

Listing 3. Seeded proposal for DevTeam (rendering of its JSON; defects marked).

```
agents: Analyst{} Architect{} Developer{exec} Tester{}
      -- D5
writes: Analyst:Req Architect:Arch Developer:Code, Test
       Tester:Test -- D3
goal:   (Criterion, c.story.status='accepted', checked)
rule Story2Operation { from s:Req!UserStory (s.status='
accepted')
  to op:Arch!Operation(name<-s.title, story<-s,
    signature<-@llm('Derive an API signature', s.title)
    @check compiles) }
rule Op2Edit { from op:Arch!Operation to e:Code!CodeEdit(
  body<-@llm('Implement', {op.signature, op.returnType
  }) -- D2
  @check compiles) }
rule Edit2TestRun { from e:Code!CodeEdit to r:Test!TestRun
  ( -- D1
    verdict<-@llm('Run and judge', e.body) @check runs) }
done: cover(G), valid, fresh, noObl, Tester.says('ALL
TESTS PASS') -- D4
```

B. Step B: six well-formedness conditions

The checker decides six conditions on Θ alone, with no task data, LLM call or execution. Two graphs support them. The type graph $\mathcal{G}_\Theta$ has an edge $C \rightarrow D$ for every rule with C in its source pattern that creates D . The feature-level data-flow graph $\mathcal{F}_\Theta$ has an edge $(C, f) \rightarrow (D, g)$ whenever a binding that writes feature g of D reads feature f of C : in its expression, its footprint, or an argument that its validator reads (op.story.criteria reads Operation.story, UserStory.criteria and the attributes of Criterion). Let B be the set of features written by stochastic bindings with a behavioural validator.

W1, hand-offs are well typed: Every rule of T reads only classes of $src(T)$ and creates a class of $tgt(T)$; every bound feature exists and its expression type-conforms; every footprint and validator-argument path type-checks against $src(T)$; stochastic bindings target attributes and have at least one validator. Rules out the formal part of D2: op.returnType in Listing 3 names information no one produces.

W2, one writer per artefact: The sets $\omega(a)$ are pairwise disjoint and cover $V \setminus \{MM_0\}$, and each class is created by exactly one rule. Rules out D3: Developer and Tester both write Test.

W3, targets are complete: Every mandatory feature of a created class is bound exactly once, and every structural reference binding is resolvable: the source class it yields equals the reference type, or some rule maps it there. Rules out the content part of D2: no consumer receives an empty mandatory slot or a dangling link.

W4, every obligation reaches a check that reads it: (a) For each $(C, s, checked) \in G$, some feature of C reaches B in $\mathcal{F}_\Theta$ (anchored coverage); for each $(C, s, delivered) \in$

G , the declared deliverable feature is in B , references C , and is reachable from a feature of C . (b) Every view is reachable from MM_0 in the hand-off graph and reaches a view containing a feature of B . Rules out D1: in Listing 3 no binding reads any feature of Criterion. A path in $\mathcal{G}_\Theta$ is not enough: a builder can create one test per criterion whose oracle never sees the criterion (?). Guards are not decidable in general; the checker warns when a guard on an obligation’s flow differs from its scope, and the runtime clause $cover(G)$ enforces coverage per instance.

W5, the engine decides completion: φ is exactly the engine clauses $cover(G)$ (every in-scope goal object has a validated descendant, and a validated deliverable if delivered), *valid*, *fresh* and *noObl*, and the hand-off graph is acyclic [13]. Rules out D4: `Tester.says(...)` is not engine-checkable.

W6, work goes to agents that can do it: For every stochastic binding b of a hand-off whose target view agent a owns, $\tau(b) \subseteq \kappa(a)$. Rules out D5: runs needs code execution, which the Tester lacks.

Proposition 1 (Cost). W1–W6 are decidable in time polynomial in $|\Theta|$ (classes, features, rules, bindings, path segments).

Sketch. W1 navigates each path once; W2, W3, W6 compare finite sets; W4 is reachability in $\mathcal{F}_\Theta$, whose size is bounded by bindings times path segments, and in the hand-off graph; the cycle test of W5 is a topological sort.

Proposition 2 (Admission guarantees). If Θ is admitted and the runtime stops with φ true, then (i) every in-scope goal object has a descendant whose value passed a behavioural validator, and for delivered obligations a validated deliverable, whose inputs read data derived from that object; (ii) no object has two producers and no view two writers; (iii) every LLM-written value was produced from a recorded footprint under a stamp and passed its validator; (iv) completion was decided by engine clauses only.

Sketch. (i) combines W4(a) at type level with ATL’s matched-rule semantics and $cover(G)$ at instance level; (ii) is W2; (iii) is W1 with the runtime; (iv) is W5. Not guaranteed: that the forms capture what the task needs, that validators are strong enough, or that agents reason correctly.

C. Step C: diagnostics and admission

A failed condition yields a diagnostic: the condition, the offending element, and how to fix it. Listing 4 is the checker’s actual output on Listing 3: one violation per seeded defect and nothing else. Diagnostics are complete for the conditions checked and name a rule, path or write right rather than “improve coordination”. Algorithm 1 feeds them back to the builder for at most k_{adm} rounds.

Listing 4. Checker output on the seeded proposal.

Algorithm 1 Admission (Steps A–C)

Require: task x , builder LLM $\mathcal{B}$, budget k_{adm}

- 1: $MM_0, M_0 \leftarrow \text{Lift}(x) \triangleright$ goal view and model, deterministic
- 2: $\Theta \leftarrow \mathcal{B}.\text{Propose}(x, MM_0)$; $i \leftarrow 0$
- 3: loop
- 4: $diag \leftarrow \emptyset$
- 5: for $c \in \{W1, \dots, W6\}$ do
- 6: $diag \leftarrow diag \cup \text{Violations}(c, \Theta, \mathcal{G}_\Theta, \mathcal{F}_\Theta)$
- 7: if $diag = \emptyset$ then return $\text{Compile}(\Theta, M_0) \triangleright$ admitted
- 8: if $i = k_{adm}$ then return $\text{NotAdmitted}(diag)$
- 9: $\Theta \leftarrow \mathcal{B}.\text{Revise}(\Theta, diag)$; $i \leftarrow i + 1$

W1	Op2Edit.body: footprint path 'op.returnType': Arch! Operation has no feature 'returnType' (its features are ['name', ' signature', 'story'])
W2	view Test written by ['Developer', 'Tester']; needs exactly one writer
W4	goal obligation (Criterion, ...): no behavioural validator reads data derived from Criterion
W5	done-clause Tester.says('ALL TESTS PASS') is not engine- checkable
W6	Edit2TestRun.verdict needs ['exec']; owner Tester has []
REJECTED: 5 violation(s)	

D. Steps D–F: run, attribute, repair

An admitted team is compiled onto the unchanged AgentM2M runtime: views become metamodels, hand-offs become rule modules with @check expressions over the validator library, and the task is lifted into the goal model. Algorithm 2 runs it to a fixpoint and evaluates φ , then attributes and repairs failures.

Locate: A failed *valid* or *noObl* clause names a pair (target object, stochastic binding); through the trace this gives the rule, the hand-off, the owning agent and the stamped footprint. A failed *cover(G)* clause names a goal object with no validated descendant. Failures that are consequences of a located one (the coverage gap of the same goal object, stale stamps behind an escalation) are not attributed again. Location is a lookup, not an inference over a transcript.

Classify: Because the footprint is recorded and small, the engine replays the binding (lines 6–10): a validator that gives two verdicts on the same value is a validator fault; if a replay on the same footprint passes, a sampling fault; if a replay on the footprint widened by one hop (every attribute of each source variable and of the objects its references reach) passes, a footprint fault, an instance of D2; if re-sampling an LLM-written value in the footprint lets the binding pass, an upstream fault, attributed to that upstream binding and its agent; otherwise a specification fault. With r replays per step the procedure makes at most $2r + 2r$ LLM calls per failure.

Repair: Each fault class has a remedy, and every team change is checked before it is applied (line 13): sampling faults get a fresh budget; upstream faults turn the upstream value into an obligation; footprint faults widen the footprint by the paths that made the replay pass;

Algorithm 2 Run, attribute and repair (Steps D–F)

Require: admitted Θ , goal model M_0 , budgets k (samples), k_{rep} , r

- 1: for $j = 0, \dots, k_{rep}$ do
- 2: $fails \leftarrow \text{RunToFixpoint}(\Theta, M_0, k)$; if φ then return Done
- 3: if $j = k_{rep}$ then return Failed($fails$)
- 4: for $f \in \text{Roots}(fails)$ do $\triangleright$ drop consequential failures
- 5: $(t, b, \rho, a, fp) \leftarrow \text{Locate}(f, TL)$ $\triangleright$ trace lookup
- 6: if $chk_b(v) \neq chk_b(v)$ then $c \leftarrow \text{validator}$
- 7: else if $\exists$ pass among r samples on fp then $c \leftarrow \text{sampling}$
- 8: else if $\exists$ pass among r samples on $\text{Widen}(fp)$ then $c \leftarrow \text{footprint}$
- 9: else if $\exists u \in fp$ written by b' : re-sampling b' lets b pass then $c \leftarrow \text{upstream}(b')$
- 10: else $c \leftarrow \text{specification}$
- 11: $report \leftarrow report \cup \{(c, t, b, \rho, a)\}$
- 12: $\Delta \leftarrow \text{Remedy}(report) \cup \mathcal{B}.\text{ProposeDelta}(\Theta, report)$
- 13: if $\text{Check}(\Theta \oplus \Delta) = \emptyset$ then $\Theta \leftarrow \text{ApplyInPlace}(\Theta, \Delta)$

specification and coverage faults go back to the builder, which proposes a team delta Δ . An admitted delta is applied to the running team: rule-level edits regenerate one module and invalidate exactly the stamps whose prompt, validator or footprint changed; additive edits (new views, agents, hand-offs) are a HOT; anything else rebuilds the team. Accepted values outside the change are kept, which a free-form builder, rebuilding from scratch, cannot do.

What AutoM2M does not solve: The checker verifies that a team fits together, not that its forms capture the task: an essential concept missing from the goal view is invisible to it. W4 demands a behavioural validator but not a strong one; weak validators turn wrong values into accepted ones. And errors inside one agent’s reasoning are outside any composition check; at best they are classified as sampling or specification faults.

IV. Related Work

Automatic team construction: Team builders generate roles, plans, state machines or workflow code per task: CaptainAgent, AutoAgents, AgentVerse, DyLAN, EvoAgent, MetaAgent and MAS-Zero [2], [3], [14]–[18], and offline searches over workflows and topologies such as ADAS, AFlow, GPTSwarm and MaAS [19]–[22]. Their quality signal is an end-to-end score, an LLM critic or observer, or execution feedback; MetaAgent’s finite state machine makes control explicit but not the content passed between states [17], and self-improving builders recover feedback from traces as textual gradients [23]. AutoM2M does not propose a new search strategy; it proposes a target representation that any of them could emit and a check that any of them could run inside its loop. Hand-designed software teams (MetaGPT, ChatDev, self-collaboration) [24]–[26] fix roles and hand-offs by hand; He, Treude and Lo survey the area [27].

Failure analysis and attribution: MAST locates most multi-agent failures in system design and inter-agent misalignment [28]; AgentAsk places them on hand-off edges and intervenes with clarification questions [29]; Kim et al. show that whether a team helps depends on whether its structure fits the task [30]; Lu et al. report that frameworks complete about half of programmable tasks and that communication is rarely analysed [31]. Who&When and TraceElephant attribute failures from transcripts and find agent- and step-level attribution unreliable [9], [32]. AutoM2M attacks the same hand-offs statically and makes attribution a lookup plus a bounded replay over recorded footprints.

Structured communication: MetaGPT’s documents, DSPy signatures, schema-validated shared state (PatchBoard), Agora’s protocol documents, and MCP/A2A make individual messages well formed or reliably transported [24], [33]–[37]; La Malfa et al. argue that LLM teams ignore classical coordination protocols [38], which KQML-style languages provided [39]. These approaches check messages. AutoM2M checks the team: coverage of obligations, single writers, complete targets, engine-decided completion and capabilities, across all hand-offs.

Software architecture and agent-oriented SE: Architectural mismatch [40], [41], design by contract [42] and formal connectors [43] motivate our conditions, and Gaia’s role schemas already stated responsibilities, permissions and protocols [44]. Ad hoc teamwork studies agents that meet without prior coordination [45], [46]; a team builder is in the opposite position, since it creates all teammates and can coordinate them in advance.

Model-driven engineering: AutoM2M reuses ATL’s execution model and typing [6], persisted traceability [7], [8], incremental execution [47], HOTs [10], models@run.time [11], and termination results for transformation networks [13], [48]. Work on LLM-generated domain models and model completion studies the quality of single generated models [49]–[51]; we check how generated models and transformations fit together, a well-formedness problem for transformation networks whose authors are LLMs and whose values are stochastic.

V. Evaluation

[TODO: evaluation]

VI. Conclusion

[TODO: conclusion]

Data availability. [TODO: artifact]

References

- [1] Anonymous, “AgentM2M: M2M transformations for agentic collaborations in software engineering,” 2026, under review; anonymised for double-anonymous review. Prototype and pilot data in the anonymous artifact.
- [2] L. Song, J. Liu, J. Zhang, S. Zhang, A. Luo, S. Wang, Q. Wu, and C. Wang, “Adaptive in-conversation team building for language model agents,” arXiv preprint arXiv:2405.19425, 2024.

- [3] G. Chen, S. Dong, Y. Shu, G. Zhang, J. Sesay, B. F. Karlsson, J. Fu, and Y. Shi, "AutoAgents: A framework for automatic agent generation," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2024, pp. 22–30.
- [4] H. Klare, M. E. Kramer, M. Langhammer, D. Werle, E. Burger, and R. Reussner, "Enabling consistency in view-based system development—the Vitruvius approach," *Journal of Systems and Software*, vol. 171, p. 110815, 2021.
- [5] H. Bruneliere, E. Burger, J. Cabot, and M. Wimmer, "A feature-based survey of model view approaches," *Software and Systems Modeling*, vol. 18, no. 3, pp. 1931–1952, 2019.
- [6] F. Jouault, F. Allilaire, J. Bézivin, and I. Kurtev, "ATL: A model transformation tool," *Science of Computer Programming*, vol. 72, no. 1–2, pp. 31–39, 2008.
- [7] F. Jouault, "Loosely coupled traceability for ATL," in *Proceedings of the European Conference on Model Driven Architecture Workshop on Traceability*, vol. 91, Nuremberg, Germany, 2005, pp. 29–37.
- [8] J. M. Vara, V. A. Bollati, Á. Jiménez, and E. Marcos, "Dealing with traceability in the MDD of model transformations," *IEEE Transactions on Software Engineering*, vol. 40, no. 6, pp. 555–583, 2014.
- [9] M. Chen, J. Wang, F. Mu, Y. Wang, Z. Liu, H. Feng, and Q. Wang, "Seeing the whole elephant: A benchmark for failure attribution in LLM-based multi-agent systems," *arXiv preprint arXiv:2604.22708*, 2026.
- [10] M. Tisi, F. Jouault, P. Fraternali, S. Ceri, and J. Bézivin, "On the use of higher-order model transformations," in *Model Driven Architecture – Foundations and Applications*, ser. *Lecture Notes in Computer Science*, vol. 5562. Springer, 2009, pp. 18–33.
- [11] G. Blair, N. Bencomo, and R. B. France, "Models@run.time," *Computer*, vol. 42, no. 10, pp. 22–27, 2009.
- [12] B. Li, W. Wu, Z. Tang, L. Shi, J. Yang, J. Li, S. Yao, C. Qian, B. Hui, Q. Zhang, Z. Yu, H. Du, P. Yang, D. Lin, C. Peng, and K. Chen, "Prompting large language models to tackle the full software development lifecycle: A case study," *arXiv preprint arXiv:2403.08604*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.08604>
- [13] H. Klare and J. Gleitze, "Termination and expressiveness of execution strategies for networks of bidirectional model transformations," *Formal Aspects of Computing*, vol. 35, no. 3, pp. 1–35, 2023.
- [14] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian, Y. Qin, X. Cong, R. Xie, Z. Liu, M. Sun, and J. Zhou, "AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors," in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=EHg5GDnyq1>
- [15] Z. Liu, Y. Zhang, P. Li, Y. Liu, and D. Yang, "A dynamic LLM-powered agent network for task-oriented agent collaboration," in *Proceedings of the First Conference on Language Modeling (COLM)*, 2024.
- [16] S. Yuan, K. Song, J. Chen, X. Tan, D. Li, and D. Yang, "EvoAgent: Towards automatic multi-agent generation via evolutionary algorithms," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025.
- [17] Y. Zhang, X. Liu, and C. Xiao, "MetaAgent: Automatically constructing multi-agent systems based on finite state machines," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. *Proceedings of Machine Learning Research*, vol. 267, 2025, pp. 75 667–75 694.
- [18] Z. Ke, A. Xu, Y. Ming, X.-P. Nguyen, R. Chin, C. Xiong, and S. Joty, "MAS-Zero: Designing multi-agent systems with zero supervision," in *Advances in Neural Information Processing Systems*, 2025.
- [19] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems," in *International Conference on Learning Representations*, 2025.
- [20] J. Zhang, J. Xiang, Z. Yu, F. Teng, X.-H. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, and C. Wu, "AFlow: Automating agentic workflow generation," in *International Conference on Learning Representations*, 2025.
- [21] M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber, "GPTSwarm: Language agents as optimizable graphs," in *Proceedings of the 41st International Conference on Machine Learning*, ser. *Proceedings of Machine Learning Research*, vol. 235, 2024, pp. 62 743–62 767.
- [22] G. Zhang, L. Niu, J. Fang, K. Wang, L. Bai, and X. Wang, "Multi-agent architecture search via agentic supernet," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. *Proceedings of Machine Learning Research*, vol. 267, 2025, pp. 75 834–75 852.
- [23] S. He, R.-Z. Wang, Z.-Y. Du, H.-Y. Bai, Z.-Y. Cao, Y. Cheng, and B. Zheng, "Learning to evolve: A self-improving framework for multi-agent systems via textual parameter graph optimization," *arXiv preprint arXiv:2604.20714*, 2026.
- [24] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations*, 2024.
- [25] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024, pp. 15 174–15 186.
- [26] Y. Dong, X. Jiang, Z. Jin, and G. Li, "Self-collaboration code generation via ChatGPT," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 7, pp. 1–38, 2024.
- [27] J. He, C. Treude, and D. Lo, "LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–30, 2025.
- [28] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, "Why do multi-agent LLM systems fail?" in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2025.
- [29] B. Lin, K. Yang, Z. Tan, Y. Lai, C. Zhang, G. Zhang, X. Yu, M. Yu, X. Wang, Y. Zhang, and Y. Wang, "AgentAsk: Multi-agent systems need to ask," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026.
- [30] Y. Kim, K. Gu, C. Park, S. Schmidgall, A. A. Heydari, Y. Yan, Z. Zhang, Y. Zhuang, Y. Liu, M. Malhotra, P. P. Liang, H. W. Park, Y. Yang, X. Xu, Y. Du, S. Patel, T. Althoff, D. McDuff, and X. Liu, "Towards a science of scaling agent systems," *arXiv preprint arXiv:2512.08296*, 2025.
- [31] R. Lu, Y. Li, and Y. Huo, "Exploring autonomous agents: A closer look at why they fail when completing tasks," in *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, New Ideas and Emerging Results Track, 2025.
- [32] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu, "Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. *Proceedings of Machine Learning Research*, vol. 267, 2025, pp. 76 583–76 599.
- [33] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: Compiling declarative language model calls into self-improving pipelines," in *International Conference on Learning Representations*, 2024.
- [34] S. Zhang, Y. Shi, and L. Wang, "PatchBoard: Schema-grounded state mutation for reliable and auditable LLM multi-agent collaboration," *arXiv preprint arXiv:2605.29313*, 2026. [Online]. Available: <https://arxiv.org/abs/2605.29313>
- [35] S. Marro, E. La Malfa, J. Wright, G. Li, N. Shadbolt, M. Wooldridge, and P. Torr, "A scalable communication protocol for networks of large language models," *arXiv preprint arXiv:2410.11905*, 2024.

- [36] Anthropic, “Model context protocol,” Specification, 2024. [Online]. Available: <https://modelcontextprotocol.io/specification>
- [37] Google, “Agent2Agent (A2A) protocol,” Specification, 2025. [Online]. Available: <https://a2a-protocol.org/>
- [38] E. La Malfa, G. La Malfa, S. Marro, J. M. Zhang, E. Black, M. Luck, P. Torr, and M. Wooldridge, “Large language models miss the multi-agent mark,” in *Advances in Neural Information Processing Systems (Position Paper Track)*, 2025.
- [39] T. Finin, R. Fritzson, D. McKay, and R. McEntire, “KQML as an agent communication language,” in *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM)*, 1994, pp. 456–463.
- [40] D. Garlan, R. Allen, and J. Ockerbloom, “Architectural mismatch: Why reuse is so hard,” *IEEE Software*, vol. 12, no. 6, pp. 17–26, 1995.
- [41] —, “Architectural mismatch: Why reuse is still so hard,” *IEEE Software*, vol. 26, no. 4, pp. 66–69, 2009.
- [42] B. Meyer, “Applying “design by contract”,” *Computer*, vol. 25, no. 10, pp. 40–51, 1992.
- [43] R. Allen and D. Garlan, “A formal basis for architectural connection,” *ACM Transactions on Software Engineering and Methodology*, vol. 6, no. 3, pp. 213–249, 1997.
- [44] M. Wooldridge, N. R. Jennings, and D. Kinny, “The Gaia methodology for agent-oriented analysis and design,” *Autonomous Agents and Multi-Agent Systems*, vol. 3, no. 3, pp. 285–312, 2000.
- [45] P. Stone, G. A. Kaminka, S. Kraus, and J. S. Rosenschein, “Ad hoc autonomous agent teams: Collaboration without pre-coordination,” in *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence*, 2010, pp. 1504–1509.
- [46] J. Wang, Y. Li, Y. Zhang, W. Pan, and S. Kaski, “Open ad hoc teamwork with cooperative game theory,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [47] F. Jouault and M. Tisi, “Towards incremental execution of ATL transformations,” in *Theory and Practice of Model Transformations*, ser. *Lecture Notes in Computer Science*, vol. 6142. Springer, 2010, pp. 123–137.
- [48] P. Stevens, “Maintaining consistency in networks of models: bidirectional transformations in the large,” *Software and Systems Modeling*, vol. 19, no. 1, pp. 39–65, 2020.
- [49] K. Chen, Y. Yang, B. Chen, J. A. Hernández López, G. Mussbacher, and D. Varró, “Automated domain modeling with large language models: A comparative study,” in *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, 2023, pp. 162–172.
- [50] M. Ben Chaaben, L. Burgueño, and H. A. Sahraoui, “Towards using few-shot prompt learning for automating model completion,” in *2023 ACM/IEEE 45th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, 2023, pp. 7–12.
- [51] R. Clarisó and J. Cabot, “Model-driven prompt engineering,” in *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, 2023, pp. 47–54.